model Â¶

Java ds-api counterpart in Java: `com.feedzai.commons.dsapi.Model`.

logger `module-attribute` Â¶

```
logger = getLogger(__name__)
```

Model Â¶

```
Model(pyjnius_java)
```

Bases: `JavaWrapper`

Representing a Data Science Model in Pulse.

slug `property` Â¶

```
slug: str
```

Sanitized name. Equivalent to Pulse's 'internal name'.

subclasses `class-attribute` `instance-attribute` Â¶

```
subclasses: List[JavaWrapper] = []
```

evaluate Â¶

```
evaluate(name: str, input_datasource: DataSource, model_eval_config: ModelEvalConfig, wait: bool = False, job_profile: JobProfile | None = None, job_properties: dict[str, str] | None = None, file_format: str | None = None) -> 'Evaluation'
```

Evaluate the model against input_datasource using model_eval_config, with the option of using the default job profiles, a specific job profile, or custom spark_job_properties.

Note

if both job_properties and job_profile are specified and there is intersection between their properties, the job_properties property will prevail over the job_profile property.

Parameters:

Name	Type	Description	Default
`name` Â¶	`str`	Evaluation name.	*required*
`input_datasource` Â¶	`DataSource`	DataSource to be evaluated.	*required*
`model_eval_config` Â¶	`ModelEvalConfig`	ModelEvalConfig configuration for the model evaluation.	*required*
	`bool`		`False`

Name	Type	Description	Default
`wait` Â¶		Whether to wait for the model evaluation to finish. The evaluation outcome will be logged. By default, False.	
`job_profile` Â¶	`JobProfile`	Pulse UI custom job profile.	`None`
`job_properties` Â¶	`Mapping`	Map containing spark job configurations. It will apply default configurations (Pulse Properties Profile) and job_properties.	`None`
`file_format` Â¶	`str`	Output file format. When not defined, it will reuse the format of the input data source. Possible options are: "parquet" (preferred) or "csv".	`None`

Returns:

Type	Description
`Evaluation`	The model evaluation object.

Examples:

```
 >>> app : App
>>> data = app.pullDataSourceByDesc("data")
>>> model_project = app.pullModelProjectByDesc("model_project")
>>> model = model_project.pullModelByDesc("model")
>>> job_profile = app.pullJobProfileByDesc("job_profile_name")
>>> eval_config = ModelEvalConfig()
...
>>> evaluation = model.evaluate(
...     name=str(uuid.uuid4()),
...     input_datasource=data,
...     model_eval_config=eval_config,
... )
>>> # or
>>> evaluation = model.evaluate(
...     name=str(uuid.uuid4()),
...     input_datasource=data,
...     model_eval_config=eval_config,
...     job_profile=job_profile,
... )
>>> # or
>>> evaluation = model.evaluate(
...     name=str(uuid.uuid4()),
...     input_datasource=data,
...     model_eval_config=eval_config,
...     job_properties={"spark.executor.memory": "5g"},
... )
>>> # or
>>> evaluation = model.evaluate(
...     name=str(uuid.uuid4()),
...     input_datasource=data,
...     model_eval_config=eval_config,
...     job_profile=job_profile,
...     job_properties={"spark.executor.memory": "5g"},
... )
```

export_binary Â¶

```
export_binary() -> bytes | None
```

Export the model binary bundle in a ZIP.

The result of this export is a directory with the model ID and its contents inside it. E.g. for a LightGBM model, you should expect to have something akin to:

```
<model ID>/
    LightGBM_model.txt
```

To read its contents you can unzip it by doing

```
 import io
from zipfile import ZipFile

with ZipFile(io.BytesIO(model.export_binary())) as f:
    txt = f.read(f"{model.id}/LightGBM_model.txt").decode("ascii")
```

For arbitrary models, you can search for the files in the bundle by using `ZipFile.namelist` :

```
 with ZipFile(io.BytesIO(binary)) as f:
    print(f.namelist())
```

Note

In case the binary fails to export, this method returns None.

Returns:

Type	Description
`bytes`	The model binary bundle.

get_config Â¶

```
get_config()
```

Get the training configuration from the model.

Returns:

Type	Description
`TrainingConfig`	The training config.

get_hyperparameters Â¶

```
 get_hyperparameters(ignore_defaults: bool = False, parameters_to_exclude: list[str] | set[str] |
None = None) -> dict[str, Any]
```

Returns the mapping between hyperparameters and their values.

Parameters:

Name	Type	Description	Default
`ignore_defaults` Â¶	`bool`	Whether to ignore or not hyperparameters with default values.	`False`
			`None`

Name	Type	Description	Default
`parameters_to_exclude` Â¶	`sequence of str`	Hyperparameters to exclude from the result.	

Returns:

Type	Description
`dict[str, Any]`	Hyperparameters and their values for the model.

get_ignored_fields Â¶

```
get_ignored_fields() -> set[str]
```

Return the field names ignored during model training.

get_schema Â¶

```
get_schema() -> Schema | None
```

Return the schema associated with the data source used to train this ML model.

refresh Â¶

```
refresh(datasource: DataSource, target: str | None = None, new_model_name: str | None = None, model_project: ModelProject | None = None, model_config: TrainingConfig | None = None, wait: bool = False) -> 'Model'
```

Refreshes a model with a new DataSource.

Parameters:

Name	Type	Description	Default
`datasource` Â¶	`DataSource`	The new datasource to train.	*required*
`target` Â¶	`str`	The target column, default is the same as the previous model. This is optional for supervised models, but required for unsupervised models.	`None`
`new_model_name` Â¶	`str`	The new model name, default is the model's name with suffix "_fresh" and current datetime as unix timestamp.	`None`
`model_project` Â¶	`ModelProject`	The model project, default is the same one as the model.	`None`
`model_config` Â¶	`TrainingConfig`	The model configuration, default is the same one as the model.	`None`
`wait` Â¶	`bool`	Whether to wait for the model refresh to finish. By default, False.	`False`

Returns:

Type	Description
`Model`	The new model object.

Examples:

```
 >>> data = app.pullDataSourceByDesc("data")
>>> model_project = app.pullModelProjectByDesc("model_project")
>>> model = model_project.pullModelByDesc("model")
...
>>> refreshed_model = model.refresh(data)
```